

Curso: Ciência da Computação

Semestre: 4º Semestre

Disciplina: Desenvolvimento Web

Professor: Felipe Pires

Alunas: Isabella Silva e Sena, Luísa Castro Souza e Maria Eduarda Almeida Campelo

DOCUMENTO ARQUITETURA

Aplicação Web para Organização e Acompanhamento de Leituras

1 OBJETIVO DA ARQUITETURA

A arquitetura do sistema Estante tem como objetivo definir a organização estrutural da aplicação, estabelecendo as principais camadas, componentes, responsabilidades, tecnologias e integrações necessárias para o funcionamento do sistema.

A solução será desenvolvida utilizando Python e Django como base tecnológica, com persistência em banco de dados relacional e disponibilização de uma API REST para integração com outros sistemas.

A arquitetura também contempla a integração com a API externa Open Library, utilizada para consultar informações bibliográficas de livros e auxiliar o usuário durante o cadastro de obras em sua biblioteca.

A definição antecipada da arquitetura busca promover a separação de responsabilidades, facilitar a manutenção do sistema, reduzir o acoplamento entre seus componentes e permitir sua evolução durante a implementação da Fase 2.

2 ESTILO ARQUITETURAL

O sistema Estante adotará uma arquitetura em camadas, implementada sobre o framework Django. Essa abordagem organiza a aplicação de acordo com responsabilidades específicas, permitindo que a interface, as regras de negócio, o acesso aos dados e as integrações externas sejam tratados de forma organizada.

A arquitetura será composta principalmente pelas camadas de apresentação, aplicação e dados, complementadas por componentes responsáveis pela API REST e pela integração com a API externa Open Library.

A comunicação entre as camadas seguirá uma estrutura controlada, evitando que a interface de usuário tenha acesso direto às estruturas de persistência ou às integrações externas sem passar pelos componentes responsáveis pelo processamento da aplicação.

A escolha desse modelo é adequada ao escopo do Estante por oferecer uma estrutura simples, organizada e compatível com as funcionalidades previstas, sem introduzir complexidade arquitetural desnecessária.

3 CAMADAS DA APLICAÇÃO

3.1 Camada de Apresentação

A camada de apresentação será responsável pela interação entre os usuários e o sistema. Ela apresentará as informações, formulários, mensagens e resultados das operações realizadas no Estante.

Entre as funcionalidades acessadas por essa camada estão o cadastro e autenticação de usuários, gerenciamento da biblioteca, acompanhamento das leituras, registro de sessões, gerenciamento de metas e visualização de relatórios.

As principais tecnologias previstas para essa camada são Django Templates, HTML, CSS e JavaScript.

3.2 Camada de Aplicação

A camada de aplicação será responsável por coordenar os casos de uso e executar as regras de negócio do sistema.

Nessa camada serão organizadas as operações relacionadas ao gerenciamento da biblioteca, leituras, sessões de leitura, metas anuais, relatórios e integração com serviços externos.

A separação dessa camada permite evitar que as regras de negócio fiquem concentradas diretamente nas páginas da aplicação.

3.3 Camada de Dados

A camada de dados será responsável pela representação, consulta e persistência das informações utilizadas pelo sistema.

O acesso aos dados será realizado por meio dos Models e do ORM do Django, permitindo que a aplicação trabalhe com as entidades do domínio sem depender diretamente de comandos SQL espalhados pelo código.

Entre as principais entidades previstas estão Usuário, Livro, Leitura, Sessão de Leitura e Meta Anual.

3.4 Camada de Integração

A camada de integração será responsável pela comunicação entre o Estante e serviços externos.

O principal serviço externo previsto é a Open Library API, utilizada para consulta de informações bibliográficas.

Essa camada também será responsável por tratar respostas externas e situações de indisponibilidade ou falha na comunicação.

4 COMPONENTES PRINCIPAIS

4.1 Interface Web

Responsável pela interação dos usuários com o sistema por meio de páginas web. Permite acessar as funcionalidades de cadastro, autenticação, biblioteca, leituras, sessões, metas e relatórios.

4.2 Views e URLs

Responsáveis por receber as requisições realizadas pela interface web, identificar a operação solicitada e encaminhá-la para o processamento correspondente.

4.3 Serviços da Aplicação

Responsáveis pela coordenação das operações e aplicação das regras de negócio. Esse componente concentra a lógica necessária para o funcionamento das principais funcionalidades do sistema.

Entre os serviços previstos estão:

- Serviço de Biblioteca;
- Serviço de Leitura;
- Serviço de Sessão de Leitura;
- Serviço de Meta Anual;
- Serviço de Relatórios;
- Serviço de Integração com a Open Library.

4.4 Models e ORM

Responsáveis pela representação das entidades do domínio e pela comunicação com o banco de dados por meio de Django ORM

4.5 API REST

Responsável por disponibilizar os recursos do Estante para consumidores externos por meio de uma API REST. A implementação será realizada preferencialmente utilizando Django REST Framework.

4.6 Serviço de Integração Open Library

Responsável por realizar as consultas à API externa Open Library, receber os dados bibliográficos e disponibilizá-los para o fluxo de cadastro de livros.

4.7 Banco de Dados

Responsável pela persistência dos dados da aplicação. O banco será relacional e armazenará informações referentes aos usuários, livros, leituras, sessões e metas.

5 INTEGRAÇÕES EXTERNAS

5.1 Open Library

A Open Library será utilizada como fonte externa de informações bibliográficas. O usuário poderá pesquisar uma obra e selecionar um resultado para importar seus dados para o cadastro do livro. Dessa forma, a integração estará associada a uma funcionalidade real da aplicação e reduzirá a necessidade de preenchimento manual das informações bibliográficas.

Dados previstos: Título, Autor(es), Ano de publicação, Número de páginas, Capa, Identificador da obra.

5.2 API REST

O Estante disponibilizará uma API REST própria para permitir que aplicações externas consultem e, quando autorizadas, manipulem recursos do sistema. A API será implementada utilizando o Django REST Framework e utilizará JSON como formato principal de comunicação.

6 TECNOLOGIAS

TECNOLOGIA	FINALIDADE
Python	Linguagem
Django	Framework web
Django REST Framework	implementação API REST
HTML	Estrutura das páginas
CSS	Estilização da interface
JavaScript	Interações da interface
PostgreSQL	Banco de dados relacional

Django ORM	Acesso aos dados
Open Library API	Integração bibliográfica
Git	Versionamento
GitHub	Hospedagem do código e documentação

7 FLUXO DE DADOS

7.1 Fluxo da Interface

Quando o usuário realiza uma operação pela interface web, a requisição é recebida pela aplicação Django e encaminhada pelas URLs e Views. A operação é então processada pela camada de aplicação, que consulta ou altera os dados por meio dos Models e do ORM. O resultado é posteriormente devolvido à camada de apresentação.

Leitor → Interface Web → View/URL's → Serviços → Models/ ORM → PostgreSQL

8 RESPONSABILIDADES

COMPONENTES	RESPONSABILIDADES
Interface Web	Interage com os usuários
Views/URLs	Recebimento e direcionamento das requisições
Serviços	Regras e processamento da aplicação
Models	Representação da entidades
ORM	Comunicação em banco
API REST	Exposição dos recursos
PostgreSQL	Persistência de dados
Serviço Library API	Comunicação API externa

9 SEGURANÇA

A arquitetura do Estante prevê mecanismos de segurança desde sua concepção. O acesso às funcionalidades privadas será controlado por autenticação e autorização, garantindo que os usuários tenham acesso apenas aos dados pertencentes à sua conta, conforme as regras definidas pelo sistema.

A validação das informações será realizada no lado do servidor, reduzindo a possibilidade de inserção de dados inválidos ou inconsistentes.

As credenciais e demais configurações sensíveis não serão armazenadas diretamente no código-fonte. Essas informações serão fornecidas por meio de variáveis de ambiente durante a implementação e publicação da aplicação.

As senhas dos usuários não serão armazenadas em texto puro, sendo utilizados os mecanismos de proteção fornecidos pelo Django.

Durante a Fase 2, serão incorporadas as medidas adicionais de segurança previstas na especificação do projeto, incluindo HTTPS e análises de segurança SAST e DAST.

10 DIAGRAMA UML

